2027-7-24 Linux库--动态库加载

- 1. 动态链接发生的时机
  - 1.1 从 _start 进入程序
    - 1.1.1 CPU 执行的是机器指令
    - 1.1.2 从 _start 到 main
  - 1.2 静态链接与动态链接的区别
    - 1.2.1 静态链接在程序运行前完成
    - 1.2.2 动态链接推迟到程序加载阶段
- 2. 动态库如何进入进程地址空间
  - 2.1 动态库也是需要加载的文件
    - 2.1.1 先找到文件，再建立映射
    - 2.1.2 动态库映射到共享区
  - 2.2 一个物理副本，多份虚拟映射
    - 2.2.1 多个进程共享库代码
    - 2.2.2 共享物理页不等于共享全部状态
- 3. 操作系统如何记录动态库映射
  - 3.1 从进程结构找到共享区
    - 3.1.1 task_struct、mm_struct 与 vm_area_struct
    - 3.1.2 VMA 如何关联到动态库文件
  - 3.2 动态库的实际地址在加载时才确定
    - 3.2.1 库的虚拟起始地址不是固定值
    - 3.2.2 为什么不能直接改掉所有 call 指令
- 4. GOT、PLT 与动态地址重定位
  - 4.1 GOT 保存可修改的函数地址
    - 4.1.1 代码查表，GOT 保存真正的跳转地址
    - 4.1.2 查看 ELF 中的 GOT 和 PLT
    - 4.1.3 PIC 与 GOT 的关系
  - 4.2 PLT 与延迟绑定
    - 4.2.1 函数调用先进入 PLT
    - 4.2.2 第一次调用与后续调用
    - 4.2.3 库与库之间的依赖也依赖重定位
- 5. 动态库加载的完整过程
  - 5.1 从可执行文件到 main
    - 5.1.1 加载与初始化顺序
    - 5.1.2 加载完成后库函数如何被调用
  - 5.2 动态链接的整体代价与收益

        - 5.2.1 链接工作推迟到运行阶段
        - 5.2.2 一条关系链总结
- 6. 用命令观察动态库加载
    - 6.1 查看程序依赖和 ELF 信息
        - 6.1.1 检查动态链接器与依赖
        - 6.1.2 查看 GOT、PLT 和重定位
    - 6.2 观察加载过程
        - 6.2.1 临时打印动态链接器的加载信息
        - 6.2.2 将命令输出与结构图对应

1. 动态链接发生的时机

1.1 从 `_start` 进入程序

1.1.1 CPU 执行的是机器指令

程序最终会被翻译成 CPU 能够执行的指令。反汇编看到的一行通常由地址、指令长度、操作码和操作数组成：

```
1131:  31 ed                  xor    %ebp,%ebp
1134:  49 89 d1               mov    %rdx,%r9
1137:  5e                     pop    %rsi
1138:  48 89 e2               mov    %rsp,%rdx
```

ELF Header 中记录的入口地址决定了 CPU 开始执行的位置。对 C/C++ 程序来说，直接入口通常是 `_start`，而不是用户编写的 `main`。

1.1.2 从 `_start` 到 `main`

程序启动的大致链路是：

```
_start
  -> 准备栈和运行环境
  -> 解析并加载动态库
  -> 完成符号解析与地址重定位
  -> __libc_start_main
  -> main
```

因此，在 `main` 的第一条语句执行之前，动态链接器已经完成了大量工作。程序员平时看不到这些步骤，但动态库加载失败、符号未找到或重定位错误，往往就发生在这一阶段。

1.2 静态链接与动态链接的区别

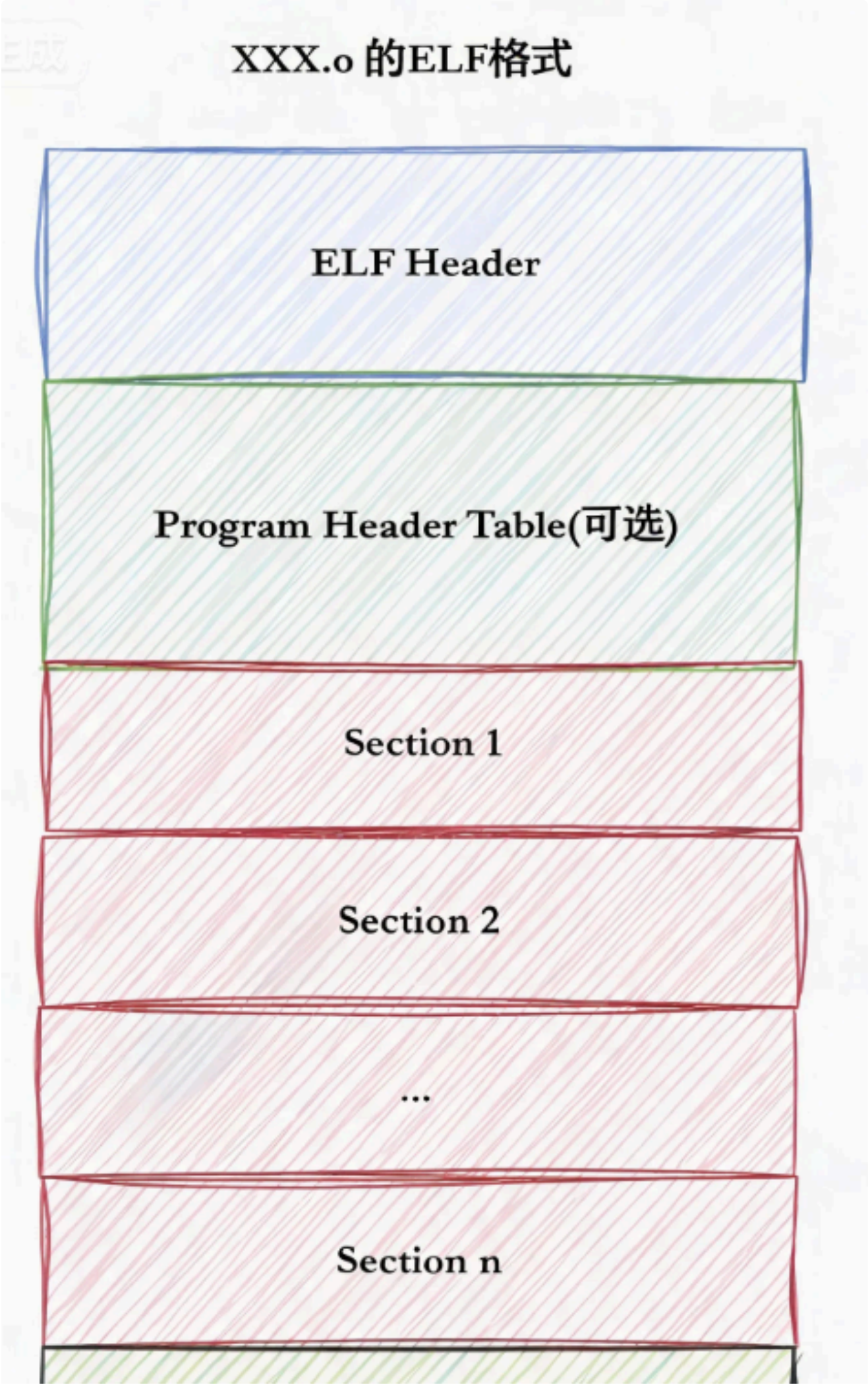
1.2.1 静态链接在程序运行前完成

静态链接会把程序实际用到的目标代码合并到可执行文件中。链接器在生成可执行文件时完成符号解析和地址修正，不需要在程序启动阶段再寻找对应的静态库。

由多个 `.o` 和静态库形成的可执行程序仍然是 ELF 文件。静态库中的代码被链接进来以后，程序运行时不再依赖原来的 `.a` 文件。

1.2.2 动态链接推迟到程序加载阶段

动态库 `.so` 本身也是 ELF 文件。程序链接时只记录“需要哪个动态库”以及相关符号和重定位信息，真正的库代码会在程序启动后由动态链接器加载并映射。


XXX.o 的ELF格式
ELF Header
Program Header Table(可选)
Section 1
Section 2
...
Section n



对比项	静态链接	动态链接
主要发生时间	编译链接阶段	程序加载和运行阶段
库代码位置	被复制到可执行文件中	保留在独立 `.so` 中
地址确定方式	链接时完成主要重定位	加载时结合库的实际地址完成重定位
多进程复用	每个程序各自包含代码	多个进程可共享同一份库代码

动态链接的主要优势是节省磁盘空间和物理内存，并允许程序通过共享库复用二进制代码。它牺牲了一定的启动时间和运行时解析开销，换取更好的共享能力与可维护性。

2. 动态库如何进入进程地址空间

2.1 动态库也是需要加载的文件

2.1.1 先找到文件，再建立映射

动态库虽然表现为程序可以直接调用的函数，但它首先仍是一个磁盘文件。加载 `.so` 的过程至少要包含：

1. 根据程序记录的依赖关系找到对应 `.so`。
2. 打开动态库文件。
3. 读取 ELF Header 和 Program Header Table。
4. 把需要加载的 segment 映射到当前进程的虚拟地址空间。
5. 确定动态库本次加载到的虚拟地址。
6. 完成符号解析和地址重定位。

查看程序的动态库依赖可以使用：

```
ldd ./main
```

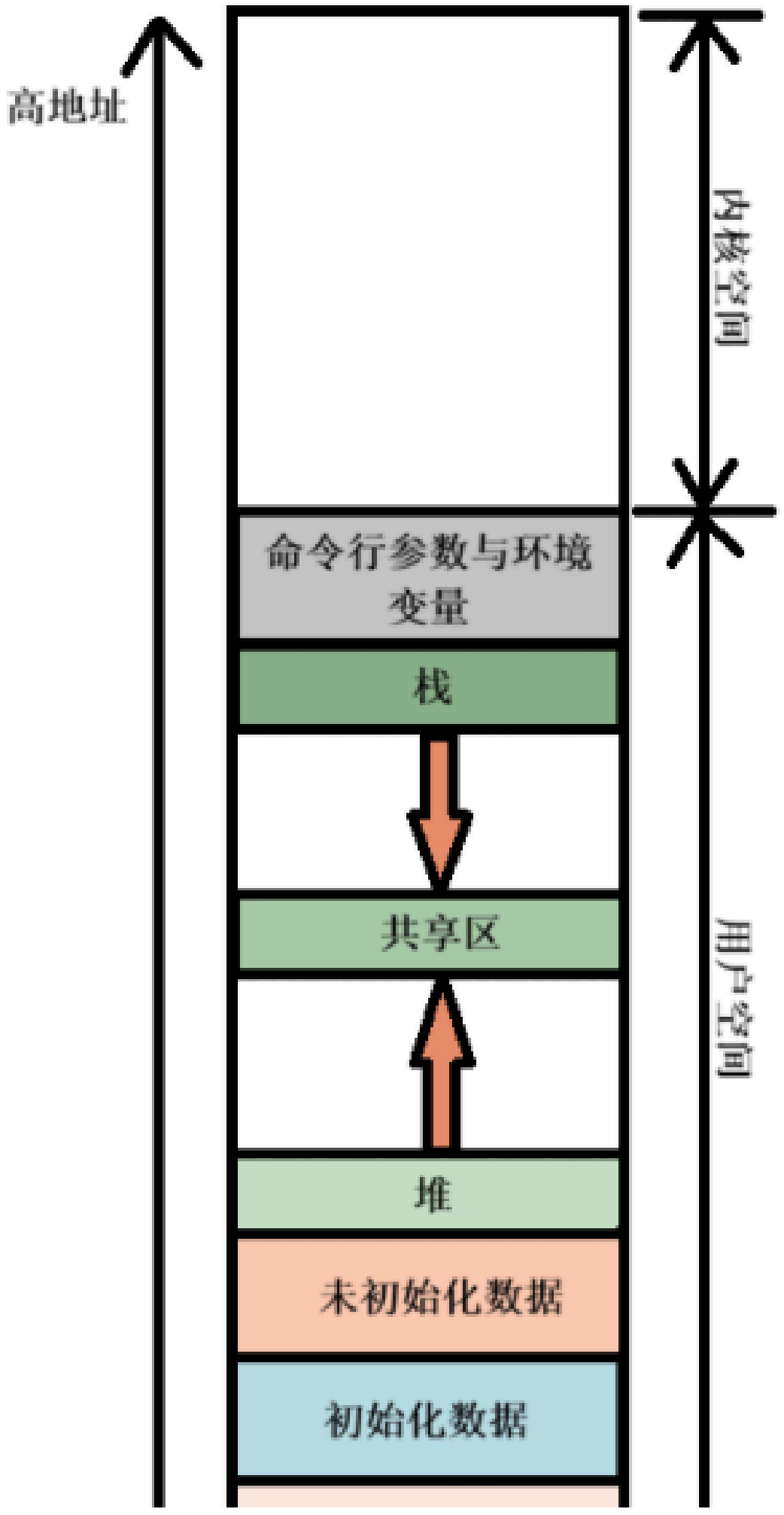
输出中通常还会出现：

```
/lib64/ld-linux-x86-64.so.2
```

它是动态链接器本身，负责加载程序依赖的共享库并完成重定位。

2.1.2 动态库映射到共享区

进程地址空间中的代码区、数据区、堆和栈有各自的位置和权限。动态库的只读代码通常不会放进程自己的代码区，而是通过文件映射进入共享区。


高地址
内核空间
用户空间
命令行参数与环境变量
栈
共享区
堆
未初始化数据
初始化数据



> 空间布局图.png 是典型的 32 位示意。x86-64 的具体地址范围不同，但“动态库代码位于共享映射区域”的思想一致。

对库函数的调用，本质上是在当前进程的虚拟地址空间内，从程序代码区跳转到共享区中的库代码，执行完成后再返回程序代码区。

2.2 一个物理副本，多份虚拟映射

2.2.1 多个进程共享库代码

当多个进程使用同一个动态库时，操作系统不需要为每个进程复制一份完整的库文件。库的文件页可以通过页缓存映射到各进程的地址空间，多个进程的页表可以指向同一批物理页面。

```
进程 A 的页表 ─┐
               ├─> 同一动态库的物理页面
进程 B 的页表 ─┘
```

这就是动态库也被称为共享库的原因：磁盘上的库可能只需要加载一次，物理内存中的只读代码页也可以被多个进程共享。

2.2.2 共享物理页不等于共享全部状态

进程之间虽然可以共享动态库的只读代码页，但以下内容仍然需要独立处理：

- 每个进程有独立的虚拟地址空间和页表。
- 动态库映射到不同进程时，虚拟起始地址不一定相同。
- 可写数据页可能使用写时复制或进程私有映射。
- GOT 中保存的是当前进程实际解析出的函数地址，因此不能直接跨进程共用。

因此，“一个物理副本”主要指库的文件内容和可共享代码页，并不是说所有进程的运行时数据都完全相同。

3. 操作系统如何记录动态库映射

3.1 从进程结构找到共享区

3.1.1 `task_struct`、`mm_struct` 与 `vm_area_struct`

Linux 描述进程和管理地址空间时，相关结构形成一条清晰的关系链：

```
task_struct
  -> mm_struct
  -> mmap
  -> vm_area_struct（栈、共享区、堆、数据区、代码区……）
```

- `task_struct` 描述进程或线程。
- `mm_struct` 描述该进程的虚拟地址空间，并维护页表等全局信息。
- `vm_area_struct` 描述一段连续的虚拟内存区域，例如共享区中的一个库映射。

3.1.2 VMA 如何关联到动态库文件

一个映射动态库的 `vm_area_struct` 通常包含：

字段	作用
`vm_start` / `vm_end`	当前虚拟内存区域的起止地址
`vm_flags`	读、写、执行等权限
`vm_file`	指向被映射的库文件
`vm_pgoff`	从文件的哪个页偏移开始映射

沿 `vm_file` 可以继续找到文件系统对象，例如 `struct file`、dentry 和 inode。加载器据此从磁盘或页缓存中取得动态库内容。

这里使用的是虚拟内存映射，而不是把整个 `.so` 简单地复制进进程地址空间。文件的只读代码页可以由多个进程映射到同一批物理页。

3.2 动态库的实际地址在加载时才确定

3.2.1 库的虚拟起始地址不是固定值

为了支持地址空间布局随机化并适应不同进程，动态库的加载基址可以变化。库中函数通常使用相对于库起点的偏移量，位置无关代码可以直接复用。

定位一个库函数时可以写成：

```
函数实际虚拟地址 = 动态库加载基址 + 函数在库中的偏移量
```

程序代码不需要在编译阶段知道某个进程最终把 `.so` 加载到了哪里。

3.2.2 为什么不能直接改掉所有 `call` 指令

编译阶段遇到动态库函数时，编译器暂时无法知道函数最终地址。若加载器直接修改可执行文件的代码区来填入地址，会受到两个限制：

- 代码区通常是只读的，不能随意修改。
- 代码页若为每个进程单独修改，就无法在多个进程间共享。

所以动态链接不会把动态库函数的绝对地址直接写进每一处可执行代码，而是把需要变化的地址集中放进可写的数据结构中，这就是 GOT。

4. GOT、PLT 与动态地址重定位

4.1 GOT 保存可修改的函数地址

4.1.1 代码查表，GOT 保存真正的跳转地址

GOT（Global Offset Table，全局偏移表）位于可写数据区域，每一项记录当前模块要访问的全局变量或函数的地址。代码仍可保持只读并共享，地址变化只影响进程自己的 GOT。

```
程序代码
  -> 读取 GOT 中的函数地址
  -> 跳转到 GOT 指向的实际库函数
```

不同进程、不同动态库的加载地址可能不同，因此每个进程需要根据当前映射结果填写自己的 GOT。

4.1.2 查看 ELF 中的 GOT 和 PLT

动态链接相关的常见 section 包括：

Section	作用
`.got`	全局偏移表
`.got.plt`	供延迟绑定使用的 GOT 区域
`.plt`	过程链接表，负责跳转到 GOT 保存的地址
`.rela.plt`	与 PLT/GOT 重定位相关的信息
`.dynamic`	动态链接器读取的依赖和重定位信息

可以查看这些 section：

```
readelf -S ./main | grep -E '\.got|\.plt|\.dynamic'
```

```
ZL@VM-0-14-ubuntu:~/code/lesson20-lib$ readelf -S a.out
There are 31 section headers, starting at offset 0x3698:

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .interp           PROGBITS         0000000000000318  00000318
       000000000000001c  0000000000000000   A       0     0     1
  [ 2] .note.gnu.pr[...] NOTE             0000000000000338  00000338
       0000000000000030  0000000000000000   A       0     0     8
  [ 3] .note.gnu.bu[...] NOTE             0000000000000368  00000368
       0000000000000024  0000000000000000   A       0     0     4
  [ 4] .note.ABI-tag     NOTE             000000000000038c  0000038c
       0000000000000020  0000000000000000   A       0     0     4
  [ 5] .gnu.hash         GNU_HASH         00000000000003b0  000003b0
       0000000000000024  0000000000000000   A       6     0     8
  [ 6] .dynsym           DYNSYM           00000000000003d8  000003d8
       00000000000000a8  0000000000000018   A       7     1     8
  [ 7] .dynstr           STRTAB           0000000000000480  00000480
       000000000000008f  0000000000000000   A       0     0     1
  [ 8] .gnu.version      VERSYM           0000000000000510  00000510
       000000000000000e  0000000000000002   A       6     0     2
  [ 9] .gnu.version_r    VERNEED          0000000000000520  00000520
       0000000000000030  0000000000000000   A       7     1     8
  [10] .rela.dyn         RELA             0000000000000550  00000550
       00000000000000c0  0000000000000018   A       6     0     8
  [11] .rela.plt         RELA             0000000000000610  00000610
       0000000000000018  0000000000000018  AI       6    24     8
  [12] .init             PROGBITS         0000000000001000  00001000
       000000000000001b  0000000000000000  AX       0     0     4
```

4.1.3 PIC 与 GOT 的关系

编译动态库时使用：

```
gcc -fPIC -c library.c
gcc -shared -o liblibrary.so library.o
```

`-fPIC` 表示生成位置无关代码（Position-Independent Code）。它允许同一份库代码被映射到不同进程的不同虚拟地址，而不需要修改代码区。

可以把它概括为：

```
PIC = 相对编址 + GOT
```

代码通过相对寻址找到 GOT，再从 GOT 中取得当前进程对应的实际函数地址。

4.2 PLT 与延迟绑定

4.2.1 函数调用先进入 PLT

程序调用动态库函数时，常见的机器码形式是先跳到 `函数名@plt`：

```
call puts@plt
```

PLT（Procedure Linkage Table，过程链接表）是代码与 GOT 之间的跳板。`puts@plt` 根据 GOT 中的地址继续跳转到真正的 `puts` 实现。

4.2.2 第一次调用与后续调用

延迟绑定的典型过程如下：

1. 程序第一次调用动态库函数，进入对应的 PLT 表项。
2. 此时 GOT 表项尚未保存最终函数地址，而是指向动态链接器的解析代码。
3. 解析代码查找真实函数，例如 `puts`。
4. 解析成功后，把真实函数地址写入该进程的 GOT 表项。
5. 后续再次调用时，PLT 从 GOT 中直接取得地址并跳转，不再重复完整解析。

延迟绑定减少了程序启动时对所有动态符号进行重定位的开销，因为有些库函数在本次运行中可能根本不会被执行。

某些系统或安全配置采用立即绑定或完整 RELRO，会在加载时提前解析全部或部分符号。因此，PLT/GOT 的基本结构不变，但“第一次调用才解析”并非所有配置下的必然行为。

4.2.3 库与库之间的依赖也依赖重定位

不仅可执行程序会调用动态库，动态库也可能依赖其他动态库。每个参与动态链接的 ELF 模块都可能带有 GOT 和重定位信息。

解析依赖关系时，动态链接器不仅需要找到各个 `.so`，还需要为它们完成符号绑定和地址重定位。多个库最终能够通过统一机制互相调用，也是它们都采用 ELF 格式的重要原因。

5. 动态库加载的完整过程

5.1 从可执行文件到 `main`

5.1.1 加载与初始化顺序

动态程序启动可以按下面的顺序理解：

1. 创建进程相关数据结构，建立 `mm_struct` 和初始虚拟内存区域。
2. 打开可执行 ELF 文件，读取 ELF Header，并通过 Program Header Table 找到需要加载的 segment。
3. 将可执行程序的代码、数据等 segment 映射到进程地址空间。
4. 如果存在 `PT_INTERP`，启动其中指定的动态链接器。
5. 动态链接器读取程序的动态依赖，找到并映射需要的 `.so`。
6. 递归处理动态库之间的依赖关系。
7. 完成符号解析和 GOT 等重定位工作。
8. 运行时库完成初始化，调用 `__libc_start_main`。
9. `__libc_start_main` 最终调用 `main`。

```
ZL@VM-0-14-ubuntu:~/code/lesson20-lib$ readelf -h a.out
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1060
  Start of program headers:          64 (bytes into file)
  Start of section headers:          13976 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

5.1.2 加载完成后库函数如何被调用

加载完成后，当前进程已经具备：

- 动态库对应的 VMA 和页表映射。
- 动态库本次加载的虚拟基址。
- 库函数相对于基址的固定偏移。
- 已经填写或等待延迟绑定的 GOT 表项。

调用动态库函数时，CPU 先在当前进程的共享映射中找到库代码，再通过 PLT/GOT 完成跳转。整个调用仍在当前进程的虚拟地址空间内进行，不需要切换到库独立创建的另一个“进程”。

5.2 动态链接的整体代价与收益

5.2.1 链接工作推迟到运行阶段

动态链接把部分符号查找和地址重定位工作从编译链接阶段推迟到了程序加载阶段。为了减少这部分开销，系统使用缓存、位置无关代码、GOT 和延迟绑定等机制。

虽然启动和首次调用存在额外成本，但换来了更少的重复存储、更低的物理内存占用以及更方便的共享库更新。

5.2.2 一条关系链总结

```
可执行 ELF 的依赖信息
  -> 动态链接器找到并打开 .so
  -> 为进程创建 VMA
  -> 通过页表把 .so 映射到共享区
  -> 得到库加载基址
  -> 基址 + 方法偏移得到函数地址
  -> 通过 PLT/GOT 完成调用
```

6. 用命令观察动态库加载

6.1 查看程序依赖和 ELF 信息

6.1.1 检查动态链接器与依赖

```
file ./main
ldd ./main
readelf -l ./main
```

重点观察：

输出	含义
`dynamically linked`	程序采用动态链接
`INTERP`	动态链接器的路径
`DYNAMIC`	动态链接和重定位信息的段
`libc.so.6 => ...`	程序实际使用的动态库及其路径

6.1.2 查看 GOT、PLT 和重定位

```
readelf -S ./main | grep -E '\.got|\.plt|\.dynamic'
readelf -r ./main
objdump -d ./main | grep -A8 '<puts@plt>'
```

这些命令可以分别查看：

- `.got`、`.plt` 等 section 是否存在。

- 哪些动态符号需要重定位。
- `puts@plt` 如何读取 GOT 并间接跳转。

6.2 观察加载过程

6.2.1 临时打印动态链接器的加载信息

```
LD_DEBUG=libs ./main
```

输出会显示动态链接器搜索、加载依赖库以及最终找到的共享库路径。该调试信息较多，适合临时观察，不适合长期开启。

6.2.2 将命令输出与结构图对应

可以把观察结果按下列关系连接起来：

```
ldd                  -> 程序依赖哪些动态库
readelf -l            -> 哪些段需要映射，动态链接器在哪里
readelf -S            -> GOT、PLT、dynamic 等 section 在哪里
readelf -r            -> 哪些符号需要动态重定位
objdump -d            -> 程序如何通过 PLT 跳入动态库
/proc/<pid>/maps      -> 动态库最终映射到当前进程的哪个虚拟地址范围
```

这样，磁盘上的 `.so` 文件、进程中的 VMA、页表映射、GOT 表项和最终的库函数调用就形成了一条完整链路。